

好, 现在看看 `public` 成员变量。为什么不采用它呢?

让我们从语法一致性开始 (同时请见条款 18)。如果成员变量不是 `public`, 客户唯一能够访问对象的办法就是通过成员函数。如果 `public` 接口内的每样东西都是函数, 客户就不需要在打算访问 `class` 成员时迷惑地试着记住是否该使用小括号 (圆括号)。他们只要做就是了, 因为每样东西都是函数。就生命而言, 这至少可以省下许多搔首弄耳的时间。

或许你不认为一致性的理由足以令人信服, 那么这个事实如何: 使用函数可以让你对成员变量的处理有更精确的控制。如果你令成员变量为 `public`, 每个人都可以读写它, 但如果你以函数取得或设定其值, 你就可以实现出“不准访问”、“只读访问”以及“读写访问”。见鬼了, 你甚至可以实现“惟写访问”, 如果你想要的话:

```
class AccessLevels {
public:
    ...
    int getReadOnly() const { return readOnly; }
    void setReadWrite(int value) { readWrite = value; }
    int getReadWrite() const { return readWrite; }
    void setWriteOnly(int value) { writeOnly = value; }
private:
    int noAccess;           //对此 int 无任何访问动作
    int readOnly;           //对此 int 做只读访问 (read-only access)
    int readWrite;          //对此 int 做读写访问 (read-write access)
    int writeOnly;          //对此 int 做惟写访问 (write-only access)
};
```

如此细微地划分访问控制颇有必要, 因为许多成员变量应该被隐藏起来。每个成员变量都需要一个 `getter` 函数和 `setter` 函数毕竟罕见。

还是不够说服你? 是端出大口径武器的时候了: 封装啦。如果你通过函数访问成员变量, 日后可改以某个计算替换这个成员变量, 而 `class` 客户一点也不会知道 `class` 的内部实现已经起了变化。

举个例子，假设你正在写一个自动测速程序，当汽车通过，其速度便被计算并填入一个速度收集器内：

```
class SpeedDataCollection {  
    ...  
public:  
    void addValue(int speed);           //添加一笔新数据  
    double averageSoFar() const;       //返回平均速度  
    ...  
};
```

现在让我们考虑成员函数 `averageSoFar`。做法之一是在 `class` 内设计一个成员变量，记录至今以来所有速度的平均值。当 `averageSoFar` 被调用，只需返回那个成员变量就好。另一个做法是令 `averageSoFar` 每次被调用时重新计算平均值，此函数有权力调取收集器内的每一笔速度值。

上述第一种做法（随时保持平均值）会使每一个 `SpeedDataCollection` 对象变大，因为你必须为用来存放目前平均值、累积总量、数据点数的每一个成员变量分配空间。然而 `averageSoFar` 却可因此而十分高效；它可以只是一个返回目前平均值的 `inline` 函数（见条款 30）。相反地，“被询问才计算平均值”会使得 `averageSoFar` 执行较慢，但每一个 `SpeedDataCollection` 对象比较小。

谁说得出哪一个比较好？在一部内存吃紧的机器上（例如一台嵌入式路边侦测装置），或是在一个并不常常需要平均值的应用程序中，“每次需要时才计算”或许是比较好的解法。但在一个频繁需要平均值的应用程序中，如果反应速度非常重要，内存不是重点，这时候“随时维持一个当下平均值”往往更好一些。重点是，由于通过成员函数来访问平均值（也就是封装了它），你得以替换不同的实现方式（以及其他你可能想到的东西），客户最多只需重新编译。（如果遵循条款 31 所描述的技术，你甚至可以消除重新编译的不便性。）

将成员变量隐藏在函数接口的背后，可以为“所有可能的实现”提供弹性。例如这可使得成员变量被读或被写时轻松通知其他对象、可以验证 `class` 的约束条件以及函

数的前提和事后状态、可以在多线程环境中执行同步控制……等等。来自 Delphi 和 C# 阵营的 C++ 程序员应该知道, 这般能力等价于其他语言中的 "properties", 尽管额外需要一组小括号。

封装的重要性比你最初见到它时还重要。如果你对客户隐藏成员变量 (也就是封装它们), 你可以确保 class 的约束条件总是会获得维护, 因为只有成员函数可以影响它们。犹有进者, 你保留了日后变更实现的权利。如果你不隐藏它们, 你很快会发现, 即使拥有 class 原始码, 改变任何 public 事物的能力还是极端受到束缚, 因为那会破坏太多客户码。Public 意味不封装, 而几乎可以说, 不封装意味不可改变, 特别是对被广泛使用的 classes 而言。被广泛使用的 classes 是最需要封装的一个族群, 因为它们最能够从“改采用一个较佳实现版本”中获益。

protected 成员变量的论点十分类似。实际上它和 public 成员变量的论点相同, 虽然或许最初看起来不是一回事。“语法一致性”和“细微划分之访问控制”等理由显然也适用于 protected 数据, 就像对 public 一样适用。但封装呢? protected 成员变量的封装性是不是高过 public 成员变量? 答案令人惊讶: 并非如此。

条款 23 会告诉你, 某些东西的封装性与“当其内容改变时可能造成的代码破坏量”成反比。因此, 成员变量的封装性与“成员变量的内容改变时所破坏的代码数量”成反比。所谓改变, 也许是从 class 中移除它 (或许这有利于计算, 就像上述的 `averageSoFar`)。

假设我们有一个 public 成员变量, 而我们最终取消了它。多少代码可能会被破坏呢? 唔, 所有使用它的客户码都会被破坏, 而那是一个不可知的大量。因此 public 成员变量完全没有封装性。假设我们有一个 protected 成员变量, 而我们最终取消了它, 有多少代码被破坏? 唔, 所有使用它的 derived classes 都会被破坏, 那往往也是个不可知的大量。因此, protected 成员变量就像 public 成员变量一样缺乏封装性, 因为在这两种情况下, 如果成员变量被改变, 都会有不可预知的大量代码受到破坏。虽然这个结论有点违反直观, 但经验丰富的程序库作者会告诉你, 它是真的。一旦你将一个成员变量声明为 public 或 protected 而客户开始使用它, 就很难改变那个成员变量所涉及的一切。太多代码需要重写、重新测试、重新编写文档、重新编译。从封装的角度观

之，其实只有两种访问权限：`private`（提供封装）和其他（不提供封装）。

请记住

- 切记将成员变量声明为 `private`。这可赋予客户访问数据的一致性、可细微划分访问控制、允诺约束条件获得保证，并提供 `class` 作者以充分的实现弹性。
- `protected` 并不比 `public` 更具封装性。

条款 23：宁以 `non-member`、`non-friend` 替换 `member` 函数

Prefer `non-member non-friend functions` to `member functions`.

想象有个 `class` 用来表示网页浏览器。这样的 `class` 可能提供的众多函数中，有一些用来清除下载元素高速缓存区（`cache of downloaded elements`）、清除访问过的 `URLs` 的历史记录（`history of visited URLs`）、以及移除系统中的所有 `cookies`：

```
class WebBrowser {
public:
    ...
    void clearCache( );
    void clearHistory( );
    void removeCookies( );
    ...
};
```

许多用户会想一整个执行所有这些动作，因此 `WebBrowser` 也提供这样一个函数：

```
class WebBrowser {
public:
    ...
    void clearEverything( );           //调用 clearCache, clearHistory,
                                      //和 removeCookies
    ...
};
```

当然，这一机能也可由一个 `non-member` 函数调用适当的 `member` 函数而提供出来：

```
void clearBrowser(WebBrowser& wb)
{
    wb.clearCache();
    wb.clearHistory();
    wb.removeCookies( );
}
```